LAPORAN PRAKTIKUM

- **Identitas Praktikum**

Nama MK	: Praktikum Pemrograman Berorientasi Objek
Kode MK	: CAK2KAB4
Bobot SKS	: 1 SKS
Tempat	: L-MM
Hari, tanggal	: Jumat, 10 April 2026
Jam	: 12:30-15:30
Topik praktikum	: Modul-5

- **Identitas Mahasiswa**

Nama lengkap	: Renisa Assyifa Putri
NIM	: 103112400123
Program Studi	: S-1 Teknik Informatika

- **Algoritma / studi kasus (jika ada) & Penjelasan**

Modul 5 memodelkan sebuah sistem informasi akademik sederhana yang terdiri dari tiga entitas: Fakultas, Program Studi (Prodi), dan Dosen. Sistem harus memenuhi dua aturan utama: (1) Prodi berada di dalam Fakultas, dan (2) Dosen tetap ada walaupun tidak terdaftar di Prodi manapun. Dari dua aturan tersebut dapat disimpulkan tiga jenis relasi antar kelas yang akan diimplementasikan menggunakan paradigma Pemrograman Berorientasi Objek (PBO).

Analisis Relasi Antar Kelas

1. **Fakultas → Prodi (Komposisi)**

   Relasi strong containment. Prodi hanya dapat lahir dari Fakultas, objek Prodi dibuat di dalam method milik Fakultas, bukan diinjeksi dari luar. Akibatnya, jika Fakultas dihapus dari memori, seluruh Prodi yang dikandungnya juga ikut musnah.

2. **Prodi → Dosen (Agregasi)**

   Relasi weak containment. Prodi hanya menyimpan referensi (pointer) ke objek Dosen yang sudah ada. Objek Dosen diciptakan di luar Prodi, lalu ditambahkan via method tambahDosen(). Satu objek Dosen yang sama boleh direferensi oleh lebih dari satu Prodi tanpa perlu duplikasi, dan jika Prodi dihapus, objek Dosen tetap utuh.

3. **Dosen ↔ Prodi (Asosiasi)**

   Relasi paling longgar. Dosen memanggil method yang menerima objek Prodi sebagai parameter (method mengajar(Prodi)). Dosen cukup tau tipe Prodi untuk sementara saat method dijalankan, tanpa menjadikan Prodi sebagai atribut. Pola ini persis seperti contoh method giveScore(Mahasiswa) pada modul 5.2.1.

Algoritma Eksekusi

1. **Alokasi memori entitas independen.** Tiga objek Dosen diciptakan langsung lewat new Dosen(...). Objek-objek ini eksis secara mandiri di heap sebelum terikat ke struktur manapun.
2. **Inisialisasi entitas induk.** Instans Fakultas dibuat sebagai container tertinggi dalam hierarki.
3. **Eksekusi relasi Komposisi.** Method tambahProdi(String) dipanggil pada Fakultas. Di dalam method ini dieksekusi new Prodi(...), sehingga siklus hidup Prodi terkunci pada Fakultas pemiliknya.
4. **Eksekusi relasi Agregasi.** Referensi objek Dosen disuntikkan ke ArrayList<Dosen> milik Prodi via tambahDosen(). Hal ini dibuktikan ketika dosen1 dipetakan ke prodiTI dan prodiSI sekaligus tanpa menduplikasi objeknya.
5. **Eksekusi relasi Asosiasi.** Method mengajar(prodi) pada Dosen dipanggil dengan memberi objek Prodi sebagai parameter. Dosen mengambil nilai dari prodi.getNamaProdi() tetapi tidak menyimpan referensi ke Prodi tersebut.
6. **Traversal hierarkis.** Method tampilkanData() pada Fakultas memicu iterasi bersarang: Fakultas → daftar Prodi → daftar Dosen, menghasilkan output terminal seperti pada bagian Hasil Running Program.

▪ Kode Program & Penjelasan

Dosen.java

```
public class Dosen {
    private String namaDosen;
    private String nip;

    public Dosen(String namaDosen, String nip) {
        this.namaDosen = namaDosen;
        this.nip = nip;
    }

    public void mengajar(Prodi prodi) {
        System.out.println("Dosen " + this.namaDosen
                + " sedang mengajar di Program Studi " + prodi.getNamaProdi());
    }

    public String getNama() {
        return this.namaDosen;
    }

    public String getNip() {
        return this.nip;
    }
}
```

Penjelasan:
Blueprint dasar entitas Dosen. Enkapsulasi diterapkan melalui modifier private pada kedua atribut (namaDosen dan nip), sehingga akses data hanya bisa lewat getter getNama() dan getNip(). Method mengajar(Prodi prodi) merupakan kunci pembuktian relasi Asosiasi: Dosen memanggil method milik Prodi (prodi.getNamaProdi()) tanpa menjadikan Prodi sebagai atribut. Pola ini identik dengan contoh pada modul 5.2.1 (method giveScore(Mahasiswa s)).

Prodi.java

```
import java.util.ArrayList;

public class Prodi {
    private String namaProdi;
    private ArrayList<Dosen> daftarDosen;

    public Prodi(String namaProdi) {
        this.namaProdi = namaProdi;
        this.daftarDosen = new ArrayList<>();
    }

    public void tambahDosen(Dosen dosen) {
        this.daftarDosen.add(dosen);
    }

    public String getNamaProdi() {
        return this.namaProdi;
    }

    public void tampilkanData() {
        System.out.println("  Program Studi: " + this.namaProdi);
        System.out.println("  Daftar Dosen Pengajar:");
        if (daftarDosen.isEmpty()) {
            System.out.println("    - (Belum ada dosen terdaftar)");
        } else {
            for (Dosen d : daftarDosen) {
                System.out.println("    - " + d.getNama() + " (NIP: " +
d.getNip() + ")");
            }
        }
    }
}
```

Penjelasan:
Implementasi relasi Agregasi. Atribut ArrayList<Dosen> daftarDosen berperan sebagai kontainer referensi (pointer) ke objek Dosen yang sudah ada. Konstruktor Prodi hanya menerima String namaProdi, tidak memaksa Dosen dibuat bersamaan. Ini sesuai aturan sistem: Dosen tetap ada walaupun tidak terdaftar di Prodi. Method tampilkanData() bertanggung jawab atas iterasi pencetakan dosen yang terdaftar di Prodi bersangkutan.

Fakultas.java

```
import java.util.ArrayList;

public class Fakultas {
    private String namaFakultas;
    private ArrayList<Prodi> daftarProdi;

    public Fakultas(String namaFakultas) {
        this.namaFakultas = namaFakultas;
        this.daftarProdi = new ArrayList<>();
    }

    public void tambahProdi(String namaProdi) {
        Prodi prodiBaru = new Prodi(namaProdi);
        this.daftarProdi.add(prodiBaru);
    }

    public Prodi getProdi(int index) {
        return this.daftarProdi.get(index);
    }

    public void tampilkanData() {
        System.out.println("Fakultas: " + this.namaFakultas);
        for (Prodi p : daftarProdi) {
            p.tampilkanData();
        }
    }
}
```

Penjelasan:

Implementasi relasi Komposisi ketat. Method tambahProdi(String namaProdi) menyembunyikan proses instansiasi Prodi dari dunia luar; class pemanggil hanya menyediakan nama prodi, sedangkan new Prodi(...) dieksekusi secara internal oleh Fakultas. Dengan pola ini, tidak mungkin menyuntikkan objek Prodi yang dibuat di tempat lain ke dalam Fakultas, keterikatan eksistensial antara Fakultas dan Prodi menjadi mutlak.

Main.java

```
public class Main {
    public static void main(String[] args) {
        // 1. Membuat objek Dosen - eksis secara independen di memori
        //    (sesuai aturan: "Dosen tetap ada walaupun tidak terdaftar di
Prodi")
        Dosen dosen1 = new Dosen("Nadia Tambunan", "198501012010121001");
        Dosen dosen2 = new Dosen("Renisa Assyifa Putri", "198702022011122002");
        Dosen dosen3 = new Dosen("Abu Abdirrahman Humaid",
"198903032012123003");

        // 2. Membuat objek Fakultas sebagai induk hierarki
        Fakultas fasilkom = new Fakultas("Ilmu Komputer");
```

```
        // 3. KOMPOSISI: Prodi diciptakan DI DALAM Fakultas
        fasilkom.tambahProdi("Teknik Informatika");
        fasilkom.tambahProdi("Software Engineering");

        // 4. AGREGASI: mengambil referensi Prodi, lalu menambahkan Dosen
        Prodi prodiTI = fasilkom.getProdi(0);
        Prodi prodiSE = fasilkom.getProdi(1);

        prodiTI.tambahDosen(dosen1);
        prodiTI.tambahDosen(dosen2);
        prodiSE.tambahDosen(dosen3);
        prodiSE.tambahDosen(dosen1); // dosen1 dipakai 2 Prodi -> bukti shared
reference

        // 5. Menampilkan seluruh hierarki data
        fasilkom.tampilkanData();

        // 6. ASOSIASI: Dosen memanggil method dengan parameter Prodi
        System.out.println("\n--- Demonstrasi Asosiasi Dosen dengan Prodi ---
");
        dosen1.mengajar(prodiTI);
        dosen2.mengajar(prodiTI);
        dosen3.mengajar(prodiSE);
        dosen1.mengajar(prodiSE);
    }
}
```

Penjelasan:

Entry point program yang berperan sebagai driver class. Alur di dalam main() mengatur urutan: pembuatan objek independen (Dosen), pembentukan hierarki Fakultas, Prodi melalui Komposisi, perangkaian referensi Dosen ke Prodi melalui Agregasi, serta pemanggilan method mengajar(Prodi) yang mendemonstrasikan Asosiasi. Pemetaan dosen1 ke dua Prodi sekaligus (prodiTI dan prodiSE) sengaja dilakukan untuk membuktikan bahwa Agregasi menggunakan shared reference, bukan duplikasi objek.

▪ Hasil Running Program

```
Fakultas: Ilmu Komputer
  Program Studi: Teknik Informatika
  Daftar Dosen Pengajar:
    - Nadia Tambunan (NIP: 198501012010121001)
    - Renisa Assyifa Putri (NIP: 198702022011122002)
  Program Studi: Software Engineering
  Daftar Dosen Pengajar:
    - Abu Abdirrahman Humaid (NIP: 198903032012123003)
    - Nadia Tambunan (NIP: 198501012010121001)

--- Demonstrasi Asosiasi Dosen dengan Prodi ---
Dosen Nadia Tambunan sedang mengajar di Program Studi Teknik Informatika
Dosen Renisa Assyifa Putri sedang mengajar di Program Studi Teknik Informatika
Dosen Abu Abdirrahman Humaid sedang mengajar di Program Studi Software Engineering
Dosen Nadia Tambunan sedang mengajar di Program Studi Software Engineering
```

Bagian A — Class Diagram

Class diagram sistem terdiri dari tiga kelas dengan tiga jenis relasi berbeda: Komposisi antara Fakultas–Prodi, Agregasi antara Prodi–Dosen, dan Asosiasi antara Dosen–Prodi.

1. Class Dosen

- **Atribut:** String namaDosen, String nip
- **Method:** Dosen(String, String), mengajar(Prodi), getNama(), getNip()
- **Relasi:** Asosiasi dengan Prodi melalui method mengajar(Prodi). Dosen tidak menyimpan Prodi sebagai atribut.

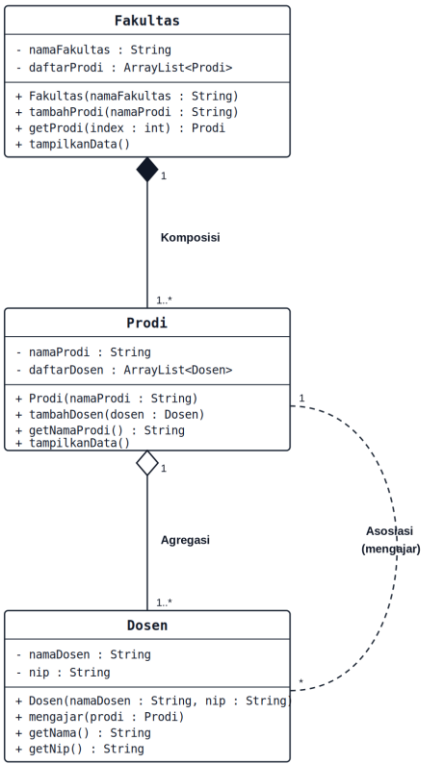
2. Class Prodi

- **Atribut:** String namaProdi, ArrayList<Dosen> daftarDosen
- **Method:** Prodi(String), tambahDosen(Dosen), getNamaProdi(), tampilkanData()
- **Relasi:** Agregasi (Prodi ◇—— Dosen). Prodi mengumpulkan referensi objek Dosen yang diciptakan di luar dirinya.

3. Class Fakultas

- **Atribut:** String namaFakultas, ArrayList<Prodi> daftarProdi
- **Method:** Fakultas(String), tambahProdi(String), getProdi(int), tampilkanData()
- **Relasi:** Komposisi (Fakultas ◆—— Prodi). Objek Prodi diinstansiasi secara eksklusif di dalam method tambahProdi() milik Fakultas.

Diagram Class UML:



Keterangan Relasi UML:

◆ Komposisi (strong) — part musnah bersama whole

◇ Agregasi (weak) — part tetap eksis walau whole hilang

- - - - - - Asosiasi — dependency via parameter method

Bagian B — Implementasi

Implementasi Java dari class diagram di atas telah berhasil menghasilkan output yang konsisten dengan seluruh aturan sistem. Tiga jenis relasi terbukti berjalan sesuai teori:

1. **Komposisi terbukti:** Fakultas "Ilmu Komputer" berhasil menciptakan Prodi "Teknik Informatika" dan "Software Engineering" secara internal. Tidak ada jalan bagi kode klien untuk menyuntikkan objek Prodi dari luar.
2. **Agregasi terbukti:** Objek dosen1 muncul di daftar dosen prodiTI sekaligus prodiSE tanpa perlu dibuat dua kali. Ini hanya mungkin terjadi ketika yang disimpan adalah referensi, bukan duplikat objek.
3. **Asosiasi terbukti:** Method mengajar(prodiTI) dan mengajar(prodiSE) pada objek dosen1 mencetak nama Prodi yang berbeda-beda, padahal Dosen sendiri tidak menyimpan Prodi sebagai atribut.